AI Stylist & Smart Wardrobe Manager -- README Documentation

Agentic AI personal styling assistant delivered over Telegram with LangGraph, Multimodal Vision, and Live RAG

1. Project Executive Summary

AI Stylist is an end-to-end agentic personal styling assistant delivered over Telegram. It transforms casual smartphone clothing photos into a structured digital wardrobe, incorporating Human-in-the-Loop (HITL) refinement, conservative dual-condition duplicate prevention, interactive body profiling, and a stateful LangGraph styling engine powered by live Open-Meteo weather, DuckDuckGo real-time fashion trends, past outfit history RAG, and deterministic color/proportion rules.

2. Judge Quickstart & Environment Setup

To evaluate the solution immediately without photographing and uploading 10+ garments, follow these quick steps to run the application with the pre-seeded evaluation wardrobe database:

```
# 1. Clone repository or extract submission code archive
git clone https://github.com/Jerrywoohoo/Fashion-sense-bot.git
cd Fashion-sense-bot

# 2. Create and activate a clean Python virtual environment (Python 3.10+ / 3.12)
python3 -m venv .venv && source .venv/bin/activate

# 3. Install production dependencies
pip install -r requirements.txt

# 4. Configure .env with your credentials (or use provided Telegram Bot Token)
cp .env.example .env
# Fill in: TELEGRAM_BOT_TOKEN (provided in telegram_bot_info.txt), AWS credentials

# 5. Launch the Bot Application
python bot.py
```

Evaluation Testing Database (wardrobe.db):

The project includes a pre-seeded evaluation SQLite database (data/wardrobe.db) with verified items across Tops, Bottoms, Outerwear, Footwear, Accessories, and logged OOTD outfits so you can test recommendations immediately.

3. Zero-Friction Judge Testing on Telegram

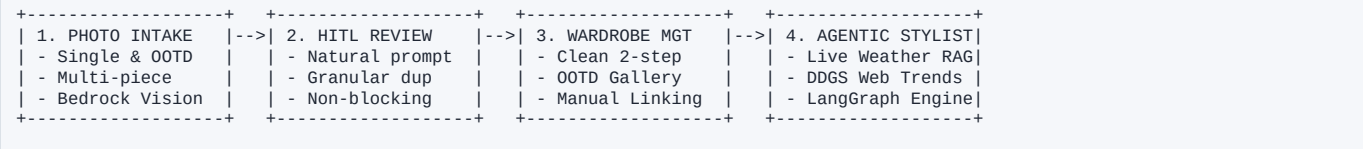
Follow these simple steps on Telegram to test the full agentic experience in under 2 minutes:

- Step 1: Open Bot: Start the bot on Telegram by searching @Fashion_sense_bot (or your configured bot) and send /start.
- Step 2: Enter Judge Pool: Send /admintest and enter evaluation password 'demo123' (or password set in your .env) to access the shared evaluation wardrobe.
- Step 3: Browse Wardrobe: Send /wardrobe to inspect cataloged items across Tops, Bottoms, Outerwear, Footwear, and Accessories with badged photo previews and 2-step management.
- Step 4: Request Styling: Send /style dinner date in Tokyo or /style rainy office day in London. The bot retrieves live weather forecasts, DuckDuckGo trend snippets, and compiles an optimal outfit.
- Step 5: Interactive Feedback: Tap 'Item in Laundry' to test rotation cooldowns, or 'More Options' to re-roll combinations based on deterministic balance rules.
- Step 6: Exit Judge Mode: Send /adminlive when finished to return to your isolated private wardrobe session.

4. Available Telegram Commands Reference

Command	Scope	Description & Usage
/admintest	Evaluation	Switches to shared test wardrobe pre-seeded with items (Password: demo123).
/adminlive	Session	Exits judge pool mode and returns to your isolated private wardrobe.
/wardrobe	Inventory	Browse cataloged pieces, badged photo previews, OOTD gallery, and laundry status.
/style [context]	Stylist	Triggers LangGraph styling engine with live weather RAG and web fashion trends.
/profile	Onboarding	7-step interactive wizard capturing body build, proportions, silhouettes, thermal preference.
/laundry	Care Mgt	View dirty laundry hamper and toggle items clean after doing laundry.
/delete [item_id]	Wardrobe	Delete individual items or execute a safe 2-step complete wardrobe reset.
/cancel	Control	Aborts any active prompt, intake correction, or conversation flow.

5. End-to-End Agentic Lifecycle



6. Detailed Methodology & System Capabilities

1. Multimodal Vision Intake (app/extractor.py):

- Single & OOTD Classification: Classifies uploaded photos into single-item photos or full multi-piece Outfit of the Day (OOTD) images.
- Rich Pydantic Extraction: Extracts structured attributes: category, sub-category, primary/accent colors, silhouette fit, fabric weight, formality tier (1-5), and styling tags.
- Visual Image Badging: Labels photos in memory with high-contrast, labeled visual badges using Pillow for crystal-clear user verification.

2. Human-in-the-Loop (HITL) & Conservative Duplicate Linking (app/handlers.py):

- Unverified Staging: Staged in unverified state (is_verified = 0) until user confirms or supplies natural-language revisions (e.g. 'the jacket is oversized charcoal wool').
- Non-Blocking Duplicate Selection: Users can toggle duplicate candidates on/off without closing the capture card, keeping item edit buttons accessible before saving.
- Dual-Condition Deduplication: Candidates must match both category and primary color hue before 64-bit pHash perceptual hashing and Bedrock LLM identity checks link them.
- Prominent Manual Duplicate Link: Tap 'Manual Duplicate Link' anytime on intake cards or directly inside /wardrobe to link pieces across photos.

3. 7-Step Profile Onboarding (app/profile_flow.py):

- Geometry & Thermal Preferences: Captures body build, vertical proportions (e.g. long torso, broad shoulders), preferred silhouettes, and thermal preference (warm/cold).

4. Contextual Agentic Stylist Graph (app/stylist_graph.py):

- Multi-Source Context RAG: Combines live Open-Meteo weather forecasts, DuckDuckGo fashion trend snippets (ddgs), past outfit history, and 48-hour anti-repeat rotation cooldown.
- Deterministic Rule Matrix: Enforces color harmony (monochromatic, complementary, analogous) and silhouette balance from app/style_matrix.py before LLM reasoning.

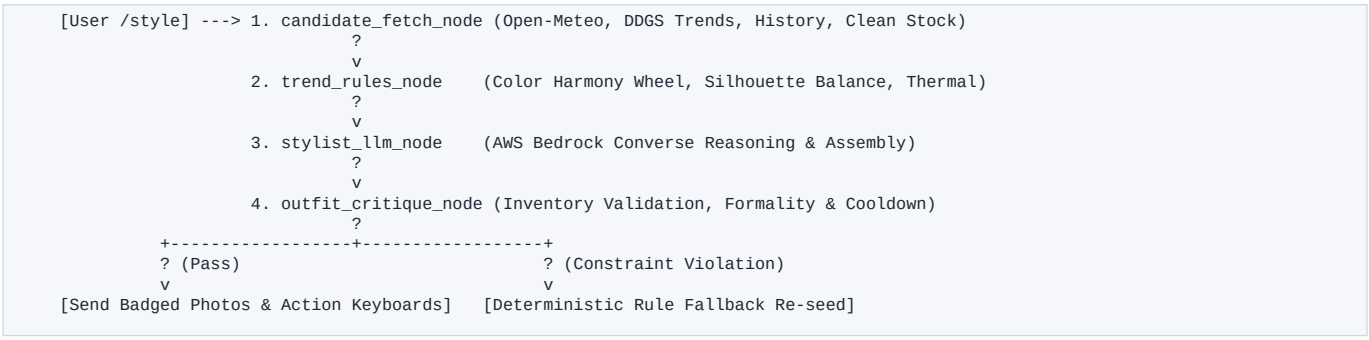
5. Wardrobe & OOTD Gallery Management (/wardrobe, /laundry):

- OOTD Gallery: Dedicated section displaying saved outfits, occasion labels, badged previews, and clear distinction between linked items and individual pieces.
- Unlinked Item Lifecycle: Deleting an OOTD cleanly removes only unlinked garments while preserving previously existing wardrobe pieces.

7. Technical Architecture & Topology

The solution decouples stateful bot routing and storage (Google Cloud Platform Compute Engine) from generative AI compute (AWS Bedrock):

8. LangGraph Stylist Node State Machine



9. Project Directory & File Purpose Map

File / Module	Layer	Purpose & Architectural Role
bot.py	Entrypoint	Configures dual HTTPX connection pools, PTB Application, handlers, and resilient polling.
app/config.py	Config	Loads .env credentials, parses Settings dataclass, validates AWS and Telegram keys.
app/database.py	Persistence	SQLite connection manager, schema migrations, CRUD queries, and wear history logging.
app/extractor.py	Vision / AI	AWS Bedrock Converse API client, vision extraction prompts, 64-bit pHash, badge renderer.
app/handlers.py	Controller	Telegram commands (/wardrobe, /style, /laundry, /delete), intake debouncing, callbacks.
app/models.py	Contracts	Pydantic v2 schemas: ExtractedGarment, UserProfile, OutfitRecommendation.
app/paths.py	Filesystem	Resolves persistent data/ directories, SQLite DB paths, and image storage paths.
app/profile_flow.py	Wizard	7-step interactive /profile ConversationHandler with fallback command routing.
app/style_matrix.py	Rule Engine	Deterministic color harmony wheel, silhouette proportion rules, offline fallback stylist.
app/stylist_graph.py	Agent Core	Stateful LangGraph orchestrator executing RAG context -> LLM stylist -> Critique loop.
app/weather.py	RAG Tool	Open-Meteo REST client retrieving real-time temperature, precipitation, and UV index.
app/web_search.py	RAG Tool	DuckDuckGo (ddgs) client retrieving live occasion-specific fashion trend snippets.
data/wardrobe.db	Data	SQLite database containing pre-seeded demo wardrobe items for judge testing.

10. Tech Stack & Dependencies

Layer	Technology	Version	Purpose
Language	Python	>= 3.10	Core asynchronous runtime
Bot Framework	python-telegram-bot	>= 21.0	Async MTProto Telegram API wrapper with polling
LLM / Vision	AWS Bedrock (Nova / Claude)	Converse API	Multimodal extraction & style reasoning
Agent Orchestration	LangGraph	>= 0.2.0	State machine with RAG & self-critique loop
Data Contracts	Pydantic	>= 2.5.0	Strict JSON schema validation and type safety
Database	SQLite 3	Built-in	Local relational storage with auto-migrations
Image Processing	Pillow (PIL)	>= 10.0.0	Resizing, 64-bit pHash, and labeled image badges
Web Trends RAG	duckduckgo-search / ddgs	>= 6.0.0	Live occasion & location fashion trend retrieval
Weather API	Open-Meteo API	REST v1	Keyless real-time weather & temperature forecasting

11. Automated Test Suite (68 Tests Passing)

All modules are validated through an automated test suite executed via 'python -m unittest discover -s . -p "test_*.py"':

- test_system_resilience.py: Verifies HTTPX polling connection timeouts, profile fallback command routing, and state resets.
- test_admin_pool.py: Verifies admin test pool isolation, password gating, and wardrobe action keyboards.
- test_batch_and_wardrobe.py: Verifies multi-photo batch intake debouncing, 4-word title display caps, and category filtering.
- test_duplicate_detection.py: Verifies 64-bit pHash calculations, hue similarity filters, and dual-condition linking.
- test_intake_flow.py: Verifies HITL verification flow, single-item edits, and wardrobe navigation.